

Universidad Tecnológica Nacional

Base de Datos II

Modelo Relacional

Autores: Ángel Simón, Laura Vélez

Índice

1. Relaciones	2
2. Tablas	2
3. Dominios	2
4. Claves	3
4.1. Claves principales	3
4.2. Otros tipos de claves	4
5. Nulos	4
6. Restricciones	4
6.1. Tipos de restricciones más comunes	5
7. Tipos de relaciones	5
7.1. Relación de uno a uno (1:1)	5
7.2. Relación de uno a varios (1:n)	6
7.3. Relación de varios a varios (n:n)	7
8. Creación de tablas por código	8
8.1. Definición de restricciones posterior a las columnas	9
8.2. Definición mediante ALTER TABLE	10
9. Claves compuestas	11
9.1. Claves foráneas compuestas	12

1. Relaciones

En el modelo relacional, cada entidad está representada lógicamente en la base de datos mediante una tabla, donde cada registro o tupla constituye una instancia particular de dicha entidad. Por ejemplo, si en una tabla de Ciudades contamos con cinco registros, dispondremos de cinco instancias distintas de ciudades, las cuales pueden pertenecer o no al mismo país, pueden ser o no capitales, entre otras características. Un principio fundamental en las bases de datos relacionales es garantizar que una tabla no almacene registros duplicados.

En la mayoría de los casos, una sola tabla resulta insuficiente para representar completamente el problema a resolver. Por ello, nuestras bases de datos contienen una colección de tablas, cada una representando una entidad específica del dominio del problema. Aunque diferentes tablas representan elementos distintos, es común que existan elementos en común entre algunas de ellas. Estos elementos compartidos establecen vínculos entre las tablas. Así, una relación se define como el vínculo existente entre dos o más entidades, describiendo generalmente algún tipo de interacción entre ellas. Por ejemplo, una relación entre las entidades Alumno y Universidad podría denominarse “estudia en”, ya que un Alumno estudia en una Universidad.

2. Tablas

La información en una base de datos se almacena dentro de tablas, las cuales se caracterizan por representarse gráficamente como objetos rectangulares compuestos por filas y columnas. Cada columna almacena información sobre una propiedad específica de la tabla (atributo), mientras que cada fila contiene una instancia particular de la tabla, conformada por los valores de cada uno de sus atributos.

3. Dominios

Los dominios determinan el conjunto de valores válidos que puede asumir un atributo determinado. Cada dominio posee un nombre descriptivo y una definición que especifica sus caracte-

atributo 1	atributo 2	atributo N
valor 1 fila 1	valor 2 fila 1	valor N fila 1
valor 1 fila 2	valor 2 fila 2	valor N fila 2
...	...	...
valor 1 fila M	valor 2 fila M	valor N fila M

Tabla 1. Representación genérica de una tabla

rísticas y restricciones.

Ejemplos de Dominios

Edad: Número entero entre 1 y 120. Ejemplos: 1, 50, 30, 68.

Nacionalidad: Texto de 50 caracteres. Ejemplos: Argentina, Chile, España, Italia.

Fecha de nacimiento: Fecha (día, mes y año). Ejemplo: 02/05/2014.

4. Claves

En el modelo relacional existen diferentes tipos de claves que permiten identificar y relacionar registros entre tablas. Las claves proporcionan información sobre si un valor representa unívocamente a un registro dentro de su propia tabla o si representa un registro de manera unívoca en otra tabla.

4.1. Claves principales

Clave primaria (PK): La clave primaria es aquella columna (o conjunto de columnas) que identifica de manera unívoca a cada registro completo. Constituye un identificador que será siempre único en toda la tabla y no acepta valores nulos.

Clave foránea (FK): La clave foránea es una columna (o conjunto de columnas) de una tabla que está relacionada y es dependiente de una clave primaria (o alternativa) en otra tabla. A diferencia de la clave primaria, puede aceptar valores nulos.

4.2. Otros tipos de claves

Clave candidata: En una tabla puede ocurrir que exista más de una columna capaz de representar unívocamente a la tabla. En tal caso, se debe elegir una para designarla como clave primaria, mientras que las demás serán consideradas claves candidatas.

Clave alternativa: La clave alternativa es cualquier clave candidata que no haya sido seleccionada como clave primaria.

Clave compuesta: Una clave compuesta es aquella que está conformada por más de una columna, siendo necesaria la combinación de todas ellas para garantizar la unicidad del registro.

5. Nulos

En ocasiones resulta necesario indicar que el contenido de un atributo equivale a "ningún valor". El problema que surge radica en que la ausencia de valor es difícil de representar de manera uniforme entre los distintos tipos de datos. Es necesario poder identificar dicha ausencia tanto en atributos de tipo entero, real, cadena de texto, booleano, fecha, entre otros. Por esta razón surge el concepto de nulo (en inglés NULL), mediante el cual podemos indicar de manera explícita que el contenido de un atributo no posee ningún valor.

En el caso de las claves foráneas, los valores nulos sirven para indicar que el registro actual no está relacionado con ningún otro registro de la tabla referenciada. En otros atributos, un valor nulo indica que dicho campo podría no ser completado por alguna razón específica del contexto.

6. Restricciones

En bases de datos, una restricción se define como una condición que debe cumplirse para permitir la incorporación o modificación de los datos. Las restricciones garantizan la integridad y consistencia de la información almacenada.

6.1. Tipos de restricciones más comunes

- **Clave primaria:** Los valores definidos como clave primaria no pueden repetirse dentro de la tabla.
- **Único (UNIQUE):** No permite que los valores de los atributos definidos de esta forma puedan repetirse.
- **Obligatoriedad (NOT NULL):** Prohíbe que el atributo definido de esta manera contenga el valor nulo.
- **Integridad referencial:** No permite ingresar valores en el atributo que no se encuentren reflejados en la tabla donde dicho atributo es clave primaria o alternativa.
- **Regla de validación (CHECK):** Condición personalizada que debe cumplirse para que un dato sea ingresado o actualizado en la tabla.

Lista 1. Principales tipos de restricciones

7. Tipos de relaciones

A continuación se presentan los tipos de relaciones que pueden existir entre diferentes tablas dentro de una base de datos. Para ilustrarlos, utilizaremos como ejemplo una base de datos empresarial que almacena información sobre empleados, departamentos, direcciones, obras sociales e idiomas.

7.1. Relación de uno a uno (1:1)

Se establece una relación de uno a uno entre una tabla A y una tabla B cuando una fila de la tabla A no puede tener más de una fila coincidente en la tabla B, y viceversa. Este tipo de relación se crea cuando ambas columnas relacionadas son claves primarias o están definidas como atributos únicos (UNIQUE).

Por ejemplo, en el caso de la relación entre Empleados y Direcciones_Empleado, se ha decidido que solo se almacenará una dirección por empleado, estableciendo así una relación uno a uno.

Uso de relaciones 1:1

Las relaciones uno a uno se utilizan cuando se desea separar información que conceptualmente pertenece a una misma entidad, generalmente por razones de organización, seguridad o rendimiento.

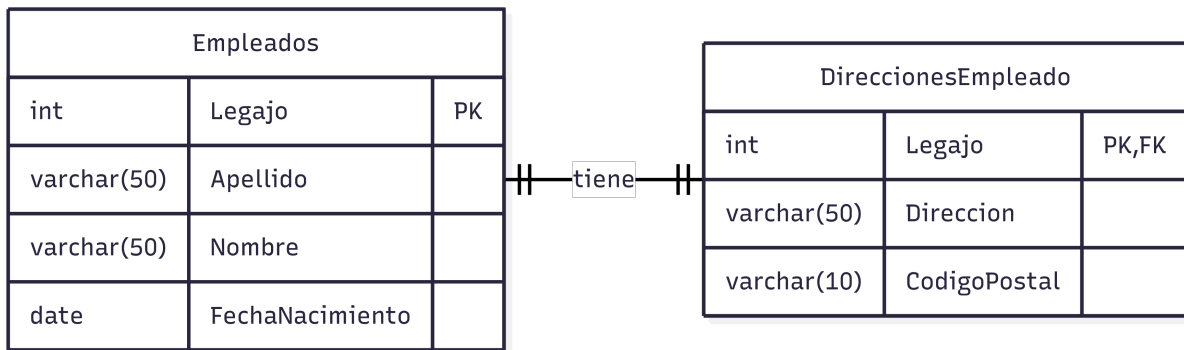


Figura 1. Diagrama de Entidad Relación de una relación 1 a 1

7.2. Relación de uno a varios (1:n)

Se dice que una tabla A tiene una relación de uno a varios con una tabla B cuando una fila de la tabla A puede corresponderse con múltiples filas de la tabla B, pero cada fila de la tabla B únicamente puede corresponderse con una fila de la tabla A. Esta relación se implementa definiendo una de las columnas como clave primaria y la otra como clave foránea.

Por ejemplo, un Area puede tener varios Empleados asignados, pero cada Empleado pertenece a una única Area. La columna IdArea en la tabla Empleados actúa como clave foránea que referencia la clave primaria de la tabla Areas.

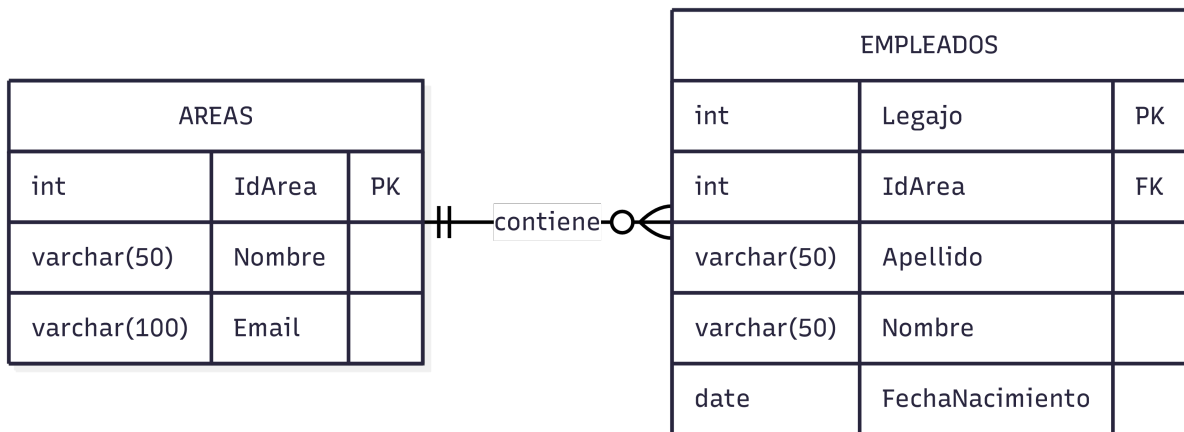


Figura 2. Diagrama de Entidad Relación de una relación 1 a Muchos

7.3. Relación de varios a varios (n:n)

En una relación de varios a varios, una fila de la tabla A puede tener muchas filas coincidentes en la tabla B, y viceversa. Este tipo de relación no puede implementarse directamente en el modelo relacional, por lo que se requiere definir una tercera tabla denominada tabla de unión o tabla intermedia. La clave primaria de esta tabla de unión está compuesta por la combinación de las claves primarias de las tablas A y B, las cuales en esta tabla intermedia actuarán como claves foráneas.

Por ejemplo, un Empleado puede hablar varios Idiomas, y un Idioma puede ser hablado por varios Empleados. Para representar esta relación se crea la tabla IdiomasEmpleados, cuya clave primaria está compuesta por Legajo (de Empleados) e IdIdioma (de Idiomas). Es común también que este tipo de tablas se denominen Entidad_X_Entidad. En este caso Idiomas_X_Empleado.

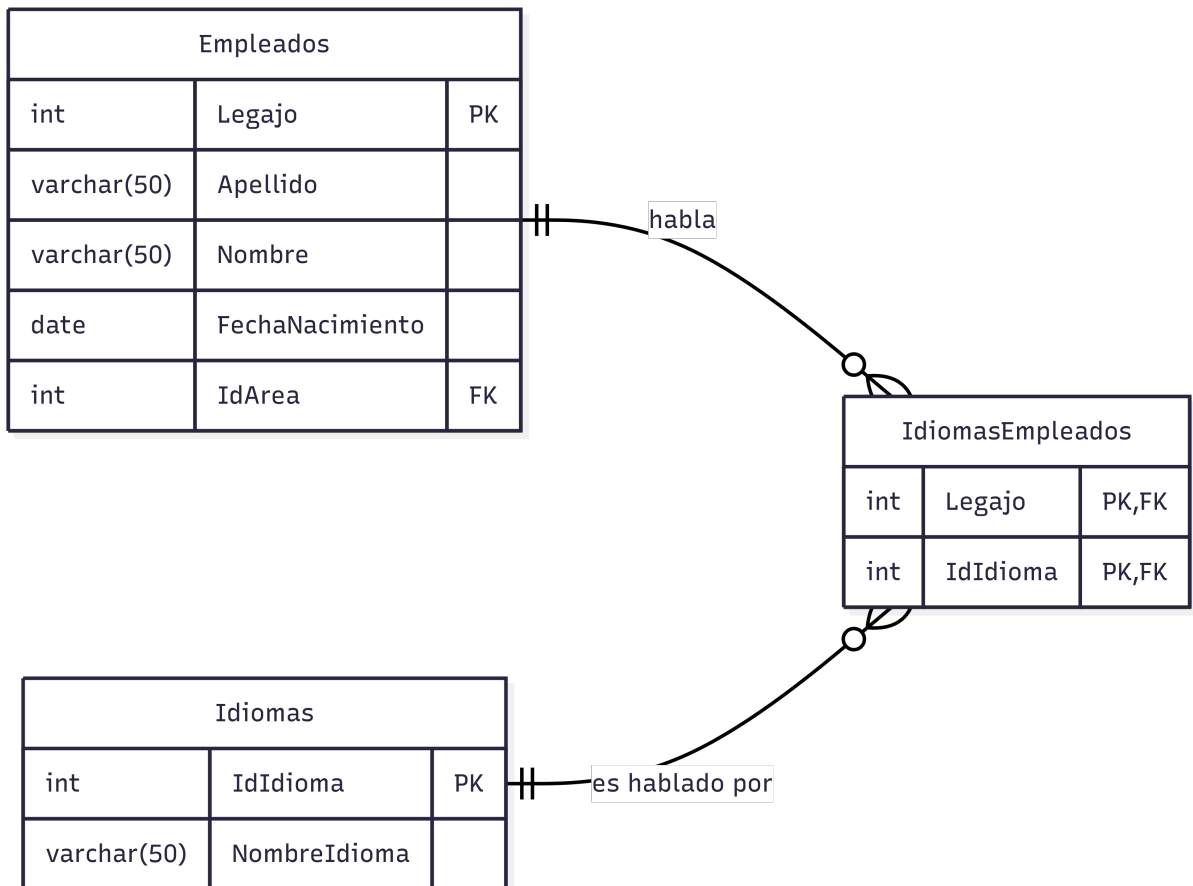


Figura 3. Diagrama de Entidad Relación de una relación Muchos a Muchos

8. Creación de tablas por código

La creación de tablas mediante código SQL permite definir con precisión la estructura de cada tabla, sus atributos y las restricciones asociadas. A continuación se presenta la sintaxis para crear la tabla Empleados.

```
CREATE TABLE Empleados(
    Legajo BIGINT NOT NULL PRIMARY KEY,
    Apellidos VARCHAR(50) NOT NULL,
    Nombres VARCHAR(50) NOT NULL,
    FechaNacimiento DATETIME NULL,
    IdArea BIGINT NOT NULL FOREIGN KEY
    REFERENCES Areas(IdArea)
)
```

Código 1. Creación de tabla con restricciones en línea

En el código anterior, tras indicar el comando `CREATE TABLE` seguido del nombre de la tabla, se definen cada una de las columnas con sus características. Para cada columna es necesario especificar un nombre y un tipo de dato, pudiendo establecer posteriormente las siguientes características:

- **NULL**: Permite que la columna contenga valores nulos.
- **NOT NULL**: Prohíbe que la columna contenga valores nulos.
- **UNIQUE**: Impide que los valores de esta columna puedan repetirse.
- **PRIMARY KEY**: Define la columna como clave primaria, cuyos valores no pueden repetirse.
- **FOREIGN KEY**: Establece que la columna es clave foránea, prohibiendo el ingreso de valores que no existan en la tabla referenciada (salvo valores nulos si están permitidos).
- **IDENTITY(inicio, incremento)**: Define una columna autonumérica que incrementa automáticamente su valor.
- **DEFAULT**: Especifica un valor por defecto cuando no se asigna uno explícitamente.

Lista 2. Especificaciones de columna en SQL

Claves foráneas

Al definir una clave foránea mediante **FOREIGN KEY**, es obligatorio indicar tanto la tabla como la columna a la que hace referencia. Es imprescindible que dicha tabla y columna hayan sido previamente creadas en la base de datos, y que la columna referenciada sea una clave primaria o única en su tabla. También deben ser consistentes los tipos de datos de las columnas a referenciar.

8.1. Definición de restricciones posterior a las columnas

Las restricciones de tipo **PRIMARY KEY** y **FOREIGN KEY** pueden establecerse en la definición de la columna siempre que no sean claves compuestas. Alternativamente, estas restricciones pueden definirse después de declarar todas las columnas:

```
CREATE TABLE Empleados(  
    Legajo BIGINT NOT NULL,  
    Apellidos VARCHAR(50) NOT NULL,  
    Nombres VARCHAR(50) NOT NULL,  
    FechaNacimiento DATETIME NULL,  
    IdArea BIGINT NOT NULL,  
    PRIMARY KEY (Legajo),  
    FOREIGN KEY (IdArea) REFERENCES Areas(IdArea)
```

```
)
```

Código 2. Restricciones definidas al final

En este ejemplo, las restricciones de tipo PK y FK se definen después de todas las columnas. Es necesario indicar entre paréntesis la columna o columnas de la tabla a las que se aplica cada restricción.

8.2. Definición mediante ALTER TABLE

Una tercera alternativa consiste en crear primero la tabla con sus columnas básicas, y posteriormente agregar las restricciones mediante sentencias ALTER TABLE:

```
CREATE TABLE Empleados(  
    Legajo BIGINT NOT NULL ,  
    Apellidos VARCHAR(50) NOT NULL ,  
    Nombres VARCHAR(50) NOT NULL ,  
    FechaNacimiento DATETIME NULL ,  
    IdArea BIGINT NOT NULL  
)  
GO  
  
ALTER TABLE Empleados  
ADD CONSTRAINT PK_Empleado PRIMARY KEY (Legajo)  
GO  
  
ALTER TABLE Empleados  
ADD CONSTRAINT FK_Empleado_Area  
FOREIGN KEY (IdArea) REFERENCES Areas(IdArea)
```

Código 3. Restricciones mediante ALTER TABLE

En este ejemplo, se ejecuta un conjunto de instrucciones DDL (Data Definition Language) para definir todas las características de la tabla. Primero se crea la tabla con la definición de cada columna, y posteriormente se altera su estructura mediante ALTER TABLE para agregar las restricciones correspondientes (ADD CONSTRAINT).

Nomenclatura de restricciones

Es fundamental asignar nombres representativos a las restricciones, como PK_EMPLEADO y FK_EMPLEADO_DEPARTAMENTO para la clave primaria y foránea respectivamente. Esto facilita su identificación y administración posterior.

Tabla Areas

En los tres ejemplos de código: 1, 2 y 3 se asume creada la tabla de Areas con su respectiva clave primaria llamada IdArea de tipo int

La instrucción GO permite ejecutar el conjunto de instrucciones como un proceso por lotes, asegurando que cada operación se complete antes de proceder con la siguiente.

9. Claves compuestas

En ciertos escenarios, la identificación unívoca de un registro requiere la combinación de múltiples atributos. Consideremos una empresa con múltiples sucursales que identifica a sus empleados mediante legajos, pero incorporando también el código de sucursal, dado que los números de legajo se repiten entre diferentes sucursales.

LEGAJO	SUCURSAL	APELLIDO y NOMBRE
1000	San Isidro	Pérez, Julio
2000	Martínez	Cáceres, Julieta
1000	Martínez	Mercado, Yamila

Tabla 2. Ejemplo de empleados con legajos repetidos en diferentes sucursales

Esta situación introduce una complejidad adicional en la estructura de la tabla. Siguiendo principios de normalización para evitar redundancia, la sucursal debe representarse mediante un código que establezca integridad referencial con una clave primaria en una tabla Sucursales.

Tras la normalización, la estructura de la tabla Empleados quedaría de la siguiente manera:

LEGAJO	IDSUCURSAL	APELLIDO y NOMBRE
1000	1	Pérez, Julio
2000	2	Cáceres, Julieta
1000	2	Mercado, Yamila

Tabla 3. Tabla normalizada con código de sucursal

En este caso, el atributo Legajo por sí solo no puede ser clave candidata, ya que no garantiza la representación unívoca de un empleado en la entidad. Sin embargo, siguiendo la lógica de identificación de la empresa, la clave candidata será la combinación de Legajo y Sucursal. Esto significa que no podrá existir más de un empleado con el mismo legajo en la misma sucursal, aunque sí pueden existir empleados con el mismo número de legajo en sucursales diferentes.

```
CREATE TABLE Empleados(  
    Legajo BIGINT NOT NULL,  
    IdSucursal BIGINT NOT NULL,  
    Apellidos VARCHAR(50) NOT NULL,  
    Nombres VARCHAR(50) NOT NULL,  
    PRIMARY KEY (Legajo, IdSucursal),  
    FOREIGN KEY (IdSucursal)  
    REFERENCES SUCURSALES(IdSucursal)  
)
```

Código 4. Tabla con clave compuesta

En el código anterior, la tabla Empleados se crea definiendo primero las columnas y posteriormente las restricciones. Para la restricción de clave primaria se indica que está conformada por dos columnas (Legajo e IdSucursal). Adicionalmente, la columna IdSucursal debe declararse como clave foránea que referencia la tabla Sucursales.

9.1. Claves foráneas compuestas

La complejidad aumenta cuando una tabla debe referenciar mediante clave foránea a otra tabla cuya clave primaria es compuesta. Consideremos el caso en que la empresa desea registrar las inasistencias de los empleados, almacenando para cada empleado la fecha en que faltó y si la ausencia fue justificada.

La tabla Inasistencias requiere nuevamente una clave compuesta. La clave primaria debe ser la combinación del empleado y la fecha de inasistencia, dado que un empleado puede faltar en diferentes fechas y en una fecha pueden faltar distintos empleados, pero no puede ocurrir que un mismo empleado registre dos inasistencias en el mismo día.

La complejidad radica en que el empleado en la tabla Inasistencias constituye una clave foránea que debe referenciar a la tabla Empleados. Como la clave primaria en Empleados está

compuesta por Legajo e IdSucursal, la clave foránea en Inasistencias también debe ser compuesta, exigiendo integridad referencial sobre la combinación de ambas columnas.

```
CREATE TABLE Inasistencias(
    Legajo BIGINT NOT NULL,
    IdSucursal BIGINT NOT NULL,
    Fecha DATE NOT NULL,
    Justificada BIT NOT NULL DEFAULT (0),
    PRIMARY KEY (Legajo, IdSucursal, Fecha),
    FOREIGN KEY (Legajo, IdSucursal)
    REFERENCES Empleados(Legajo, IdSucursal)
)
```

Código 5. Tabla con clave foránea compuesta

Integridad en claves compuestas

La clave foránea compuesta exige integridad referencial por la ocurrencia de ambos valores simultáneamente. Debe existir un registro en la tabla Empleados que contenga la combinación exacta de Legajo e IdSucursal para que se permita el ingreso de datos en la tabla Inasistencias. Ambos valores deben coincidir como conjunto.

En este ejemplo se observa una clave primaria conformada por tres columnas (Legajo, IdSucursal y Fecha) y una clave foránea conformada por dos columnas (Legajo e IdSucursal). La especificación DEFAULT en la columna Justificada establece el valor False (0) como valor por defecto cuando no se ingresa ningún valor explícitamente.

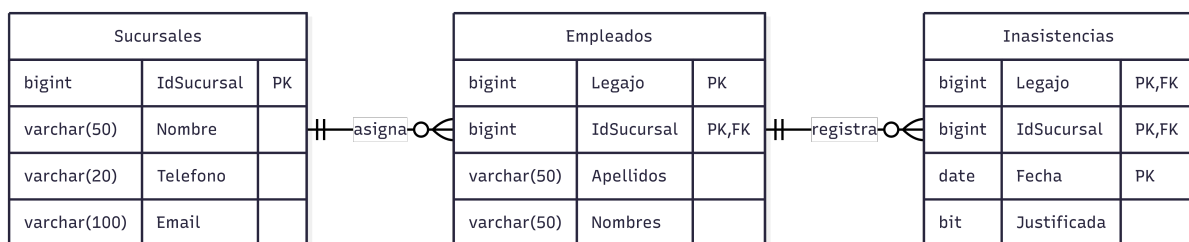


Figura 4. Diagrama de Entidad Relación de la base de datos con clave foránea compuesta